Mini Quiz Tech & AI Academy

Quality Assurance AI and Automation Testing

1. A complete teaching pathway from software foundations to practical testing of an existing website.

Wiki edition | Version 1 | September 28, 2026

Table of contents

1. What we teach 3
2. Learning outcomes and course context 4
3. Teaching sequence from beginning to end 5
4. Teaching sequence continued 6
5. How learning works in Mini Quiz AI-cademy 7
6. Learning element details continued 8
7. Lesson delivery and class preparation 9
8. Module 01 IT and quality foundations 10
9. Module 02 SDLC and early assurance 11
10. Module 03 STLC and Agile teamwork 12
11. Module 04 Test design and coverage 13
12. Module 05 Execution and defect analysis 14
13. Module 06 Web APIs and data basics 15
14. Module 07 Java programming for testers 16
15. Module 08 Git collaboration and build tools 17
16. Module 09 Selenium Playwright and Cypress 18
17. Module 10 Maintainable automation frameworks 19
18. Module 11 Karate API and integration testing 20
19. Module 12 DevOps cloud and service workflows 21
20. Module 13 Nonfunctional quality 23
21. Module 14 LLMs and Copilot foundations 24
22. Module 15 AI assisted test engineering 25
23. Module 16 Machine learning foundations 26
24. Module 17 Testing AI enabled features 27
25. Module 18 Existing website improvement project 28
26. Browser automation and Karate practice 29
27. Tool details continued 30

28. JavaScript and test execution 31
29. Docker Kubernetes AWS and ServiceNow foundations 32
30. AWS foundations and delivery evidence 33
31. ServiceNow tester foundations and ITSM 34
32. ServiceNow catalog requests and record producers 35
33. ServiceNow SLAs rules and access 36
34. ServiceNow ITAM and CMDB 37
35. ServiceNow HRSD and customer service 38
36. ServiceNow integration testing and assessment 39
37. Large language models and workplace Copilot 40
38. Copilot practice and verification 41
39. Improving the existing website for testing 42
40. Component details continued 43
41. Component details continued 44
42. Component details continued 45
43. Project scope and milestones 46
44. Assessments and completion requirements 47
45. Assessment details continued 48
46. Final project rubric and portfolio 49
47. Criterion details continued 50
48. Required portfolio 51
49. Teaching resources and curriculum coverage 52
50. Teaching tools and resources 53
51. Instructor preparation and upkeep 54

1. What we teach

1. Software planning, development, testing, release, and improvement
2. IT foundations and early quality assurance
3. Manual testing and programming
4. Web automation, APIs, and continuous integration
5. Responsible AI use
6. A testing project on the existing website
7. Review requirements and acceptance criteria early
8. Identify ambiguity and risk before implementation
9. Continue testing through development, release, and maintenance

a. One website throughout the course

1. Use the existing website throughout the course
2. Add or refine forms, tables, asynchronous messages, and API-driven results
3. Use each addition for manual testing, automation, and regression
4. No new website build is required

2. Learning outcomes and course context

a. What a learner can demonstrate at the end

1. Translate a feature request into acceptance criteria, risks, test scenarios, and traceable test cases.
2. Execute tests, investigate failures, document reproducible defects, and communicate release readiness.
3. Write browser tests with Selenium, Playwright, and Cypress, using reusable components and meaningful assertions.
4. Test APIs with Karate, run suites through Jenkins, and explain Docker, Kubernetes, AWS, and ServiceNow delivery workflows.
5. Explain LLMs and use Copilot to assist testing while checking outputs, limitations, and organizational data rules.
6. Present improvements to the existing website with a tested feature, regression evidence, and a clear handover.
7. **Audience:** beginners and career changers entering software quality assurance, plus manual testers moving into automation.
8. Delivery pace and assessment thresholds in this document are proposed settings for the course, not a fixed institutional or certification requirement.

b. Basis for this curriculum

1. This document draws on the Mini Quiz AI-cademy hub scope, the revised integration plan, the SDLC Overview worked lesson, and the existing website project.
2. The detailed module sequence, suggested duration, grading, and project minimums are curriculum recommendations added to make that scope teachable from beginning to end.

3. Teaching sequence from beginning to end

1. **Suggested duration:** 28 weeks
2. **Guided learning:** 6 hours per week, 168 hours total
3. **Independent practice:** about 4 hours per week, 112 hours total
4. Adjust delivery pace while retaining practical checkpoints

a. Foundations and automation

Weeks	Module	Main evidence
1	01 IT and quality foundations	Environment checklist and website exploration
2	02 SDLC and early assurance	Feature lifecycle and acceptance criteria
3	03 STLC and Agile teamwork	Test plan and traceability matrix
4	04 Test design and coverage	Positive negative and boundary test suite
5	05 Execution and defect analysis	Test results and reproducible defect reports
6	06 Web APIs and data basics	Browser investigation and SQL exercises
7–8	07 Java programming	Small programs and unit tests
9	08 Git and build tools	Reviewed branch and repeatable test command
10–12	09 Selenium Playwright and Cypress	Reliable browser tests for site components
13–14	10 Automation framework design	Maintainable page objects and test data

4. Teaching sequence continued

a. APIs delivery AI and the project

Weeks	Module	Main evidence
15–16	11 Karate API and integration testing	API checks and UI to data reconciliation
17–19	12 DevOps cloud and service workflows	Jenkins Docker Kubernetes AWS and ServiceNow labs
20	13 Nonfunctional quality	Accessibility usability and performance findings
21	14 LLMs and Copilot foundations	AI concepts and a verified prompt exercise
22	15 AI assisted test engineering	Reviewed AI assisted tests and defect analysis
23	16 Machine learning foundations	Dataset split metrics and error analysis
24	17 Testing AI enabled features	Evaluation set and failure analysis
25–28	18 Existing website project	Tested component improvements and portfolio

1. **After Module 6:** manual QA
2. **After Module 11:** automation
3. **After Module 13:** pipeline execution
4. **After Module 17:** AI evaluation
5. **After Module 18:** final demonstration

5. How learning works in Mini Quiz AI-cademy

a. A repeatable lesson cycle

1. Each lesson connects a short explanation to an observable action on the website.
2. **Recall:** Retrieve prior knowledge
3. **Worked example:** Study a demonstrated solution
4. **Practice:** Complete a focused exercise
5. **Explanation:** Describe the result and reasoning
6. **Review:** Revisit gaps in understanding
7. Mini quizzes support practice throughout the course; they do not replace practical assessment.

b. Notes and study guide

How we use it

1. **Objectives:** State what learners should understand or demonstrate
2. **Plain-language explanations:** Explain each concept in accessible terms
3. **Website examples:** Show how the concept applies to the existing website
4. **Lesson sources:** Record the references supporting the material

c. Mini quizzes

How we use it

1. Use short retrieval checks during class and a 25-question topic review.
2. **Answer explanations:** Describe why an answer is correct
3. **Topic tags:** Connect each question to its subject
4. **Question identifiers:** Give each question a stable reference

d. Flashcards

How we use it

1. **Severity versus priority:** Distinguish impact from urgency
2. **Verification versus validation:** Distinguish specified requirements from user needs
3. **Waits versus retries:** Distinguish readiness checks from repeated attempts

6. Learning element details continued

a. Carousels

How we use it

1. Present a visual sequence or comparison, usually 8 to 12 slides.

b. Hands on exercises

How we use it

1. **Task:** State the exercise to complete
2. **Starter material:** Supply the resources needed to begin
3. **Expected evidence:** Define what learners must submit
4. **Self-check rubric:** Help learners assess their own work
5. Learners execute and explain, not only watch.

c. Review and feedback

How we use it

1. **Incorrect answers:** Explain the reasoning error
2. **Failed tests:** Investigate the cause
3. **Misconceptions:** Correct the underlying misunderstanding
4. **Spaced practice:** Revisit earlier topics over time

7. Lesson delivery and class preparation

a. A suggested guided session

1. **Retrieval practice:** 10 minutes
2. **Teaching and demonstration:** 20 minutes
3. **Guided lab:** 35 minutes
4. **Independent variation:** 15 minutes
5. **Review:** 10 minutes
6. Continue longer labs in the next class
7. Explain the reasoning before introducing more tools

b. The first worked lesson

1. The first SDLC lesson follows a small existing feature, such as recent scores, through planning, requirements, design, development, testing, deployment, and maintenance.
2. **Requirement:** Describe the selected feature on one page
3. **Design sketch:** Show how the feature will work
4. **Test-plan outline:** Define the testing approach
5. **Release checklist:** Identify checks required before release
6. **Notes:** Explain the lesson concepts
7. **Quiz:** Review the topic with 25 questions
8. **Flashcards:** Practice 25 key recall items
9. **Carousel:** Present the lesson visually
10. **Lab:** Apply the concepts hands on
11. **Worksheet:** Record the learner's work
12. The sample is a teaching draft.
13. Review its explanations before classroom use, especially simplified claims about development models and team responsibilities.
14. The instructor adapts it to the class and records corrections.

c. Before the first class

1. Check basic computer skills, file navigation, browser use, and reading of simple instructions.
2. Provide a browser, IDE, JDK, Node.js and npm, Git, build tools, and course repository access as each module needs them.
3. Provide an instructor-approved test environment with synthetic accounts and resettable data.
4. Use local fixtures or mocks when a service is unavailable.

8. Module 01 IT and quality foundations

Quality foundations and early assurance | Week 1

a. Learning outcome

1. Explain the role of quality assurance and navigate the tools used to investigate a web application.

b. What we teach

1. **Client:** Sends requests to a service
2. **Server:** Processes requests and returns results
3. **Browser:** Displays and runs the web interface
4. **Operating system:** Hosts the application tools
5. **Files:** Store source code and test evidence
6. **Environment:** Supplies the configuration for a test run
7. **Team roles:** Identify who builds, tests, and supports software
8. **Quality attributes:** Describe what makes software fit for use
9. **Verification:** Check work against specified requirements
10. **Validation:** Check whether the result meets user needs
11. **Defect:** Identify behavior that fails an agreed expectation

c. Practical work

1. Explore the existing website
2. Map its main user flows
3. Record three quality risks
4. Inspect one request in browser developer tools

d. Evidence of learning

1. Submit an environment checklist and a feature map.
2. Explain why a successful click alone is not proof that the correct outcome occurred.

9. Module 02 SDLC and early assurance

Quality foundations and early assurance | Week 2

a. Learning outcome

1. Follow a feature from an idea to maintenance and improve its requirements before implementation.

b. What we teach

1. **Planning:** Establish the purpose and scope of a change
2. **Requirements:** Describe the behavior users need
3. **Design:** Define how the solution will work
4. **Implementation:** Build the agreed behavior
5. **Testing:** Evaluate the implementation against expectations
6. **Deployment:** Deliver the tested change
7. **Maintenance:** Support and improve the released system
8. **Agile:** Deliver and learn through short iterations
9. **Sequential models:** Organize work into ordered stages
10. **Reviews:** Find issues before implementation
11. **Testability:** Make expected behavior observable
12. **Risk:** Prioritize likely or costly failures
13. **Acceptance criteria:** State measurable completion conditions
14. **Feedback:** Revisit earlier decisions when evidence changes

c. Practical work

1. Choose a component on the existing site.
2. Write its user story and measurable acceptance criteria, identify missing decisions, and review them with a partner.

d. Evidence of learning

1. Submit the lifecycle worksheet and revised criteria.
2. Identify at least one defect that a requirements review could prevent.

10. Module 03 STLC and Agile teamwork

Quality foundations and early assurance | Week 3

a. Learning outcome

1. Plan the testing needed for a change and make coverage traceable to its requirements.

b. What we teach

1. **Requirement analysis:** Identify what needs testing
2. **Test planning:** Define the approach and scope
3. **Test design:** Specify cases and expected results
4. **Environment readiness:** Confirm the system is ready for testing
5. **Execution:** Run cases and record results
6. **Defect follow-up:** Track failures through resolution
7. **Test closure:** Summarize outcomes and remaining risks
8. **Entry criteria:** Define when testing can begin
9. **Exit criteria:** Define when testing can finish
10. **Estimation:** Plan the effort required
11. **Responsibilities:** Assign ownership of testing work
12. **Scrum events:** Coordinate delivery and feedback
13. **User stories:** Describe needs from a user perspective
14. **Definition of done:** Agree on completion expectations

c. Practical work

1. Prepare a small test plan for the chosen site component and link each acceptance criterion to a test scenario.
2. Participate in a short planning and defect-triage exercise.

d. Evidence of learning

1. Submit a test plan and traceability matrix with scope, risks, environment, data, responsibilities, and completion criteria.

11. Module 04 Test design and coverage

Manual testing and technical investigation | Week 4

a. Learning outcome

1. Select test cases deliberately and explain what each case is intended to reveal.

b. What we teach

1. **Equivalence partitions:** Group inputs with similar expected behavior
2. **Boundary values:** Test the edges of valid ranges
3. **Decision tables:** Cover combinations of conditions
4. **State transitions:** Test changes between system states
5. **Exploratory charters:** Guide focused investigation
6. **Positive tests:** Check valid use
7. **Negative tests:** Check invalid use
8. **Test oracles:** Establish how correctness is judged
9. **Risk prioritization:** Test the most significant risks first
10. **Smoke tests:** Check essential behavior quickly
11. **Regression tests:** Detect unintended changes
12. **Integration tests:** Check interactions between components
13. **Acceptance tests:** Check agreed user outcomes

c. Practical work

1. Design tests for a form, filter, or score rule.
2. Include empty input, valid and invalid values, boundaries, state changes, and recovery after an error.

d. Evidence of learning

1. **Preconditions:** State what must be true before the case
2. **Test data:** Record the inputs
3. **Steps:** Describe the actions
4. **Expected results:** Define the correct outcome
5. **Rationale:** Explain why the case matters
6. Demonstrate a decision table or state model instead of listing only happy paths.

12. Module 05 Execution and defect analysis

Manual testing and technical investigation | Week 5

a. Learning outcome

1. Run a repeatable test session and produce evidence that another person can use.

b. What we teach

1. **Execution records:** Preserve the result of each run
2. **Environment versions:** Identify the exact system tested
3. **Defect lifecycle:** Follow an issue from discovery to closure
4. **Severity:** Describe the impact of a defect
5. **Priority:** Decide how urgently to address it
6. **Reproduction steps:** Make a failure repeatable
7. **Screenshots:** Capture visible evidence
8. **Logs:** Capture system evidence
9. **Retesting:** Verify a specific fix
10. **Regression:** Check for related breakage
11. **Triage:** Decide ownership and next action
12. **Test summaries:** Communicate the overall result
13. **Release uncertainty:** State what the evidence does not establish

c. Practical work

1. Execute the Module 4 suite in the test environment.
2. Investigate a seeded or naturally occurring failure and distinguish a product defect from bad data, environment failure, or an incorrect test.

d. Evidence of learning

1. Submit results and reproducible defect reports.
2. **Expected behavior:** State what should happen
3. **Actual behavior:** State what happened
4. **Impact:** Explain who or what is affected
5. **Conditions:** Identify the environment and inputs
6. **Evidence:** Attach material that supports the finding

13. Module 06 Web APIs and data basics

Manual testing and technical investigation | Week 6

a. Learning outcome

1. Connect what the browser displays to requests, responses, and stored data.

b. What we teach

1. **HTML:** Defines page structure
2. **CSS:** Controls presentation
3. **DOM:** Represents the page as a document tree
4. **Selectors:** Identify page elements
5. **HTTP methods:** Describe the requested operation
6. **Status codes:** Communicate response outcomes
7. **JSON:** Structures exchanged data
8. **Cookies:** Store browser-side values
9. **Sessions:** Maintain interaction state
10. **Relational tables:** Organize structured records
11. **Primary keys:** Identify records uniquely
12. **Foreign keys:** Relate records across tables
13. **SELECT:** Retrieve SQL records
14. **Filtering:** Restrict query results
15. **Joins:** Combine related records
16. **Data validation:** Check stored and displayed values
17. **Document databases:** Store document-shaped records

c. Practical work

1. Inspect a site interaction, read a JSON response, and query a synthetic SQL dataset.
2. Reconcile a displayed value with its source record or mocked response.

d. Evidence of learning

1. Submit a browser investigation and SQL worksheet.
2. Explain how a correct-looking page can still contain an integration or data defect.

14. Module 07 Java programming for testers

Programming and browser automation | Weeks 7 and 8

a. Learning outcome

1. Write small readable programs and understand the code used in automation tests.

b. What we teach

1. **Variables:** Store values
2. **Types:** Describe the kind of value
3. **Operators:** Perform calculations and comparisons
4. **Conditions:** Choose a path based on a result
5. **Loops:** Repeat operations
6. **Methods:** Group reusable behavior
7. **Strings:** Represent text
8. **Arrays:** Store indexed values
9. **Collections:** Manage groups of objects
10. **Classes:** Define object structure and behavior
11. **Objects:** Represent instances of a class
12. **Encapsulation:** Control access to internal state
13. **Inheritance:** Reuse behavior through class relationships
14. **Interfaces:** Define a contract for behavior
15. **Exceptions:** Represent execution errors
16. **File handling:** Read and write test resources
17. **Assertions:** Compare actual and expected results
18. **Unit tests:** Check small pieces of behavior

c. Practical work

1. Build validators for quiz answers or form fields.
2. Predict the output of short code snippets, repair faults, and write unit tests for normal, boundary, and invalid inputs.

d. Evidence of learning

1. Submit small programs and passing unit tests with at least one explained failure.
2. Use code-based quizzes to assess understanding rather than memorized syntax alone.

15. Module 08 Git collaboration and build tools

Programming and browser automation | Week 9

a. Learning outcome

1. Share test code safely and reproduce a test run from documented instructions.

b. What we teach

1. **Repositories:** Store the project history
2. **Commits:** Record a coherent change
3. **Branches:** Isolate work in progress
4. **Pull requests:** Propose changes for review
5. **Peer review:** Evaluate another person's change
6. **Conflicts:** Reconcile overlapping edits
7. **Ignore files:** Exclude generated or local files
8. **Dependencies:** Track required libraries
9. **Project structure:** Organize code and resources
10. **Maven:** Build and run Java projects
11. **Gradle:** Compare another Java build option

c. Practical work

1. Create a branch for a test change
2. Request peer review
3. Resolve a planned conflict in the training repository
4. Run the suite with the chosen build command

d. Evidence of learning

1. Submit the reviewed change and setup instructions.
2. Another learner must be able to reproduce the run without copying the original machine configuration.

16. Module 09 Selenium Playwright and Cypress

Programming and browser automation | Weeks 10 to 12

a. Learning outcome

1. Automate the existing website with stable locators, synchronization, and meaningful checks.

b. What we teach

1. **Selenium:** Automate browsers with Java WebDriver
2. **JavaScript:** Write browser-test logic
3. **TypeScript:** Add type checking to JavaScript
4. **Playwright locators:** Identify elements by stable properties
5. **Playwright auto-waiting:** Wait for supported readiness conditions
6. **Playwright assertions:** Check expected browser state
7. **Cypress command chains:** Sequence Cypress operations
8. **Cypress retries:** Recheck supported commands and assertions
9. **Asynchronous behavior:** Handle results that arrive later
10. **Synchronization:** Align the test with application readiness
11. **Fixtures:** Supply repeatable test resources
12. **Network stubs:** Control a simulated service response
13. **Browser contexts:** Isolate browser state
14. **Failure evidence:** Capture enough detail to diagnose a failed run

c. Practical work

1. Automate a form in all three tools, then test a dynamic component.
2. Compare Selenium explicit waits, Playwright auto-waiting and assertions, and Cypress command chaining and retry behavior.

d. Evidence of learning

1. Submit a passing and deliberately failing example in each tool.
2. Explain locator and synchronization choices and select one primary framework for the larger project.

17. Module 10 Maintainable automation frameworks

Frameworks APIs and continuous integration | Weeks 13 and 14

a. Learning outcome

1. Refactor a small suite so changes are easier to diagnose and maintain.

b. What we teach

1. **Page Object Model:** Organize interactions around page components
2. **Test intent:** Separate expected behavior from interaction details
3. **JUnit:** Execute and report Java tests
4. **TestNG:** Compare another Java test runner
5. **Fixtures:** Provide shared test resources
6. **Setup:** Prepare each test
7. **Teardown:** Release resources after each test
8. **Data-driven tests:** Run a scenario with multiple inputs
9. **Configuration:** Separate environment settings from test logic
10. **Reports:** Present execution results
11. **Test independence:** Avoid dependence on execution order
12. **Cleanup:** Restore reusable test state
13. **Naming:** Make test purpose clear
14. **Flakiness analysis:** Investigate inconsistent results
15. **Retries:** Avoid concealing persistent failures

c. Practical work

1. Refactor the Module 9 tests, introduce synthetic test-data variants, and change one component to observe the cost of maintenance.
2. Run tests individually and as a suite.

d. Evidence of learning

1. Submit a documented framework and a before-and-after maintenance explanation.
2. Retries must not hide persistent failures; each failure needs evidence.

18. Module 11 Karate API and integration testing

Frameworks APIs and continuous integration | Weeks 15 and 16

a. Learning outcome

1. Validate service behavior and show agreement between the user interface and the underlying data.

b. What we teach

1. **Karate feature files:** Organize API scenarios
2. **Given When Then:** Express setup, action, and expected outcome
3. **Requests:** Define the service call
4. **Headers:** Supply request metadata
5. **Authentication:** Establish identity
6. **Authorization:** Check allowed operations
7. **Response assertions:** Check returned values
8. **Schemas:** Check response structure
9. **Match:** Compare Karate values and patterns
10. **Reusable features:** Share repeated scenario behavior
11. **Data-driven scenarios:** Apply multiple test inputs
12. **Negative cases:** Check invalid requests
13. **Repeated requests:** Evaluate repeat behavior
14. **Error handling:** Verify failure responses
15. **Mocks:** Replace unavailable services for a bounded test

c. Practical work

1. Use Karate to test a synthetic score or form-submission endpoint, then reconcile its response with the page.
2. If no suitable endpoint exists, use a documented mock behind an existing-site component.

d. Evidence of learning

1. Submit API cases and automated assertions for valid, invalid, and unavailable-service responses.
2. Record whether evidence came from a real endpoint or a mock.

19. Module 12 DevOps cloud and service workflows

Frameworks APIs and continuous integration | Weeks 17 to 19

a. Learning outcome

1. Run tests in a pipeline and use the results to support a release decision.

b. What we teach

1. **DevOps:** Coordinate development, testing, and operations
2. **Jenkins checkout:** Retrieve the intended source
3. **Jenkins build:** Produce the artifact
4. **Jenkins tests:** Execute the automated checks
5. **Jenkins reports:** Preserve run evidence
6. **Failure gates:** Stop progression after a failed check
7. **Docker images:** Package an application template
8. **Docker containers:** Run instances of an image
9. **Kubernetes Pods:** Host application containers
10. **Kubernetes Deployments:** Maintain the desired application state
11. **Kubernetes Services:** Provide stable access to Pods
12. **AWS compute:** Run workloads
13. **AWS storage:** Retain files and artifacts
14. **AWS networking:** Connect and isolate resources
15. **AWS identity:** Control resource access
16. **AWS monitoring:** Observe system behavior
17. **ServiceNow ITSM:** Test incident, problem, change, and request workflows
18. **ServiceNow catalog:** Validate request records and Record Producers
19. **ServiceNow controls:** Test SLAs, rules, notifications, and access
20. **ServiceNow data:** Validate assets and configuration relationships
21. **ServiceNow cases:** Test employee and customer service workflows
22. **ServiceNow integrations:** Verify mapping, synchronization, and failures

c. Practical work

1. Run UI and Karate tests in Jenkins
2. Launch a Docker test environment
3. Inspect a local Kubernetes deployment
4. Map the environment to basic AWS services
5. Complete the ServiceNow workflow and validation exercises in the detailed teaching sections
6. Attach test and rollback evidence

d. Evidence of learning

1. **Pipeline definition:** Submit the executable workflow
2. **Successful run:** Preserve passing execution evidence
3. **Failed run:** Show how a failure is handled
4. **Release checklist:** Record the readiness checks
5. A green build must be followed by checks of the actual intended artifact and target environment.

20. Module 13 Nonfunctional quality

Nonfunctional quality and AI assistance | Week 20

a. Learning outcome

1. Identify quality problems that functional assertions alone may miss.

b. What we teach

1. **Usability:** Evaluate how easily people complete tasks
2. **Responsive layout:** Check behavior at different screen sizes
3. **Keyboard navigation:** Operate without a mouse
4. **Labels:** Identify controls clearly
5. **Focus:** Show and manage the active control
6. **Browser coverage:** Compare supported browsers
7. **Visual comparison:** Detect presentation differences
8. **Performance baselines:** Record expected response behavior
9. **Reliability:** Assess consistent operation
10. **Security concepts:** Recognize common protection boundaries
11. **Accessibility checks:** Combine automated scans with manual investigation

c. Practical work

1. Review an existing-site form using the keyboard and a narrow viewport, measure a small approved performance scenario, and investigate a visual or browser difference.

d. Evidence of learning

1. Submit prioritized findings with conditions, evidence, and suggested remedies.
2. Keep load and security exercises within the instructor-provided test environment.

21. Module 14 LLMs and Copilot foundations

Nonfunctional quality and AI assistance | Week 21

a. Learning outcome

1. Explain what AI assistance can and cannot establish about software quality.

b. What we teach

1. **AI:** Introduce systems that perform tasks associated with intelligence
2. **Machine learning:** Learn patterns from data
3. **Generative AI:** Produce new content from learned patterns
4. **LLMs:** Generate language from supplied context
5. **Tokens:** Represent units of model input and output
6. **Training:** Change learned model parameters
7. **Inference:** Generate a result using a trained model
8. **Context:** Supply information for the current response
9. **Next-token generation:** Produce language step by step
10. **Grounding:** Supply relevant reference material
11. **Hallucinations:** Identify plausible but unsupported output
12. **Evaluation:** Judge responses against defined expectations
13. **Microsoft 365 Copilot:** Assist with workplace documents
14. **GitHub Copilot:** Assist with software development
15. **Approved access:** Follow organizational tool and data rules
16. **Model availability:** Check what the selected product permits

c. Practical work

1. Use approved Copilot access to explain a requirement or generate candidate cases; use instructor-provided outputs if access is unavailable.
2. Compare the response with the source requirement and manually identify omissions or unsupported claims.

d. Evidence of learning

1. **Prompt:** Record the submitted instruction
2. **Sanitized inputs:** Preserve the non-sensitive context
3. **Original output:** Retain the unedited AI response
4. **Corrected result:** Show the verified revision
5. Explain why fluent text and runnable code are not proof of correctness.

22. Module 15 AI assisted test engineering

Nonfunctional quality and AI assistance | Week 22

a. Learning outcome

1. Use AI to improve testing work while retaining human ownership of expected behavior and evidence.

b. What we teach

1. **GitHub Copilot:** Draft and explain code for review
2. **Microsoft 365 Copilot:** Support requirement and document work
3. **Structured prompts:** State the task and required output
4. **Case generation:** Propose tests from requirements
5. **Boundary suggestions:** Identify edge cases to review
6. **Synthetic data:** Generate nonproduction test inputs
7. **Test-code drafts:** Create code that must be executed and checked
8. **Refactoring:** Propose improvements to existing code
9. **Log summaries:** Condense failure information for review
10. **Debugging support:** Suggest explanations to verify
11. **Source checks:** Compare claims with trusted material
12. **Reproducibility:** Record enough context to repeat the work

c. Practical work

1. Apply AI assistance to one existing-site component.
2. Compare its suggested cases with a manually designed baseline, run any generated test code, and review the differences.

d. Evidence of learning

1. Submit an annotated comparison showing accepted, changed, and rejected suggestions.
2. Independently justify assertions and disclose which artifacts used AI assistance.

23. Module 16 Machine learning foundations

Machine learning AI evaluation and the final project | Week 23

a. Learning outcome

1. Interpret a basic model evaluation without confusing model performance with application correctness.

b. What we teach

1. **Features:** Represent model inputs
2. **Labels:** Represent expected target values
3. **Supervised learning:** Learn from labeled examples
4. **Unsupervised learning:** Find patterns without target labels
5. **Classification:** Predict a category
6. **Regression:** Predict a numeric value
7. **Training split:** Supply data for learning
8. **Validation split:** Support development decisions
9. **Test split:** Evaluate held-out examples
10. **Leakage:** Detect unintended access to evaluation information
11. **Class imbalance:** Recognize uneven category representation
12. **Confusion matrix:** Summarize classification outcomes
13. **Precision:** Assess the correctness of positive predictions
14. **Recall:** Assess how many actual positives were found
15. **Error metrics:** Quantify prediction errors
16. **Drift:** Observe changes in data or behavior

c. Practical work

1. Use a small synthetic dataset in a guided Python notebook or a prepared results worksheet.
2. Compare two model outputs and explain errors, class balance, and why test data must be held apart.

d. Evidence of learning

1. Submit an evaluation summary and error analysis.
2. Advanced mathematics and model training infrastructure are outside this curriculum.

24. Module 17 Testing AI enabled features

Machine learning AI evaluation and the final project | Week 24

a. Learning outcome

1. Design an evaluation for variable AI output and distinguish it from ordinary deterministic assertions.

b. What we teach

1. **Evaluation datasets:** Collect representative test inputs
2. **Reference answers:** Provide expected response examples
3. **Rubrics:** Define response-quality criteria
4. **Factuality:** Check whether claims are supported
5. **Grounding:** Check the use of supplied sources
6. **Invalid inputs:** Test unsuitable requests
7. **Prompt injection awareness:** Recognize attempts to redirect the system
8. **Privacy:** Check handling of sensitive information
9. **Repeated trials:** Observe output variability
10. **Regression evaluation:** Compare behavior after changes
11. **Latency:** Measure response time
12. **Fallbacks:** Check behavior when the model is unavailable
13. **Human review:** Resolve judgments requiring interpretation

c. Practical work

1. Evaluate an optional quiz explanation or hint component on the existing site.
2. Use recorded responses or a mock if a live model is unavailable.
3. Include correct, unsupported, irrelevant, and unavailable-response cases.

d. Evidence of learning

1. **Evaluation set:** Submit the selected test inputs
2. **Rubric:** State the quality criteria
3. **Results:** Record observed outcomes
4. **Failure analysis:** Explain the unsuccessful cases
5. **Fallback checks:** Verify unavailable-service behavior
6. Record provider and configuration only when a live model is actually used; do not claim a mock tested the model.

25. Module 18 Existing website improvement project

Machine learning AI evaluation and the final project | Weeks 25 to 28

a. Learning outcome

1. Demonstrate the complete testing workflow on useful additions to the website we already have.

b. What we teach

1. **Scope:** Bound the website changes
2. **Risks:** Prioritize significant failures
3. **Acceptance criteria:** Define expected behavior
4. **Incremental improvements:** Add testable components to the existing site
5. **Manual coverage:** Investigate behavior directly
6. **Automated coverage:** Repeat selected checks with tools
7. **Pipeline execution:** Run the checks consistently
8. **Defect correction:** Repair and retest failures
9. **Regression:** Check existing behavior after changes
10. **Release evidence:** Support the release decision
11. **Communication:** Explain results and limitations
12. **Handover:** Provide the test package and operating notes

c. Practical work

1. Choose a bounded group of components with the instructor.
2. Add or refine them in the approved test environment, then apply the course test plan, automation framework, API checks, and relevant AI-assisted or AI-feature evaluation.

d. Evidence of learning

1. Submit the completed project package and demonstrate it live.
2. **Failure:** Explain one unsuccessful result
3. **Improvement:** Demonstrate one change made after review
4. **Limitations:** State what remains unresolved
5. Site ownership, existing features, and unrelated work are preserved.

26. Browser automation and Karate practice

1. All learners use Selenium, Playwright, and Cypress.
2. We teach Java first, then the JavaScript and TypeScript needed for browser tests, including functions, objects, modules, promises, and async and await.
3. Cypress command chains need their own execution model; students do not treat them as ordinary promises.

a. Selenium

Teaching focus

1. **Java WebDriver:** Control a browser through Selenium Java code
2. **Explicit waits:** Wait until a specified browser condition is satisfied
3. **Page objects:** Encapsulate interactions with a page or component
4. **Browser lifecycle:** Start and close the browser cleanly for a test

Required evidence

1. Form and dynamic component tests with failure evidence

b. Playwright

Teaching focus

1. **TypeScript:** Add type checking to JavaScript
2. **Locators:** Identify the element a test will interact with
3. **Auto-waiting:** Wait for supported readiness conditions before acting
4. **Fixtures:** Provide shared test resources
5. **Contexts:** Isolate browser state between tests
6. **Network control:** Observe or substitute service responses during a test
7. **Traces:** Record execution details for diagnosing a failure

Required evidence

1. The same form flow plus a trace of a failed assertion

27. Tool details continued

a. Cypress

Teaching focus

1. JavaScript or TypeScript
2. **Command chains:** Sequence operations using the Cypress execution model
3. **Retry behavior:** Check what happens when an operation is repeated
4. **Intercepts:** Observe or stub network calls in a Cypress test
5. **Fixtures:** Provide shared test resources

Required evidence

1. The same form flow plus a stubbed response scenario

b. Karate

Teaching focus

1. **Feature files:** Organize Karate scenarios in readable test files
2. **HTTP requests:** Specify the operation sent to a service
3. **Match assertions:** Check expected values in a Karate response
4. **Schemas:** Check response structure
5. **Data-driven tests:** Run a scenario with multiple inputs
6. **Reuse:** Share common scenario steps across tests

Required evidence

1. **Positive API checks:** Verify valid service requests
2. **Negative API checks:** Verify invalid service requests
3. **Authorization checks:** Verify permissions for the requested operation
4. **Service-error checks:** Verify behavior when the service fails

28. JavaScript and test execution

a. JavaScript Node js and npm prerequisites

1. JavaScript variables, functions, arrays, objects, and modules
2. Asynchronous execution before Playwright and Cypress
3. **Node.js:** runtime for test tooling
4. **npm:** package management and script execution
5. TypeScript after JavaScript foundations
6. **Check installations:** node --version and npm --version
7. **Install dependencies:** npm install
8. **Reproduce locked dependencies:** npm ci
9. Read package.json and define test scripts
10. **Run defined scripts:** npm run test:playwright and npm run test:cypress
11. **Compare direct commands:** npx playwright test and npx cypress run
12. Capture the command, exit status, and report
13. Repeat the run in Jenkins

b. Depth and project expectations

1. Use one primary browser framework for the maintainable project suite after completing the comparison labs in all three.
2. Students explain the choice using application needs, team language, debugging evidence, and maintenance costs.
3. Karate remains the API automation framework for the course.
4. Run browser tests and Karate checks through documented commands and Jenkins.
5. Keep test data resettable, expected results independently justified, and results separated by tool.
6. A mock response tests the consumer behavior; it does not prove the real service works.

c. Practical checkpoint

1. Demonstrate one passing and one intentionally failing test in each browser tool.
2. Explain why a wait or retry is needed and show the evidence from a failed run.
3. Run a data-driven Karate scenario with response and schema checks.
4. Link a UI result to API evidence and clearly identify any mocked boundary.
5. **Official teaching references:** Playwright documentation https://playwright.dev/docs/intro
6. **Cypress documentation https:**//docs.cypress.io/app/get-started/why-cypress
7. **Karate documentation https:**//docs.karatelabs.io/

29. Docker Kubernetes AWS and ServiceNow foundations

1. Module 12 connects functional testing with delivery and service operations.
2. These are introductory practical skills, not administrator certification or a requirement to migrate the existing website.

a. Docker and repeatable test environments

1. **Image:** packaged application template
2. **Container:** running instance of an image
3. Dockerfiles, builds, and registries
4. Ports and environment configuration
5. Volumes, logs, and cleanup
6. Run a test service or test runner in Docker
7. Compare container behavior with a local run

b. Kubernetes and application orchestration

1. Clusters and nodes
2. Pods, Deployments, and Services
3. Desired state, readiness, and rolling updates
4. Inspect a deployment in a local cluster or guided demonstration
5. Read logs and observe a replacement Pod
6. Run a smoke test
7. Distinguish Docker packaging from Kubernetes orchestration

30. AWS foundations and delivery evidence

a. Basic cloud with AWS

1. On-demand resources, regions, and Availability Zones
2. Shared responsibility
3. IAM users and roles; least privilege
4. VPC networking and security groups
5. EC2 compute and S3 object storage
6. CloudWatch monitoring
7. **EKS:** managed Kubernetes
8. Map access, logs, and cost controls to the test environment
9. Use an instructor sandbox or an architecture worksheet.
10. Live resource creation is optional; when used, apply budgets, scoped access, and cleanup.
11. Evidence must say whether the learner ran a service or only designed the deployment.

b. Delivery evidence

1. **Docker evidence:** Record a successful run and cleanup.
2. **Kubernetes evidence:** Record a deployment observation and smoke test.
3. **AWS evidence:** Submit an environment map.
4. **Jenkins evidence:** Preserve success and failure reports.
5. **References:** Docker https://docs.docker.com/get-started/docker-overview/
6. **AWS https:**//aws.amazon.com/what-is-cloud-computing/
7. **Kubernetes on AWS https:**//aws.amazon.com/kubernetes/

31. ServiceNow tester foundations and ITSM

a. Learning objectives

1. **Platform purpose:** Explain how ServiceNow manages records and business workflows.
2. **Tester role:** Turn requirements into checks of data and complete business processes.
3. **Module map:** Distinguish ITSM, ITAM, CMDB, HRSD, and CSM.
4. **ITOM awareness:** Recognize IT Operations Management in the platform map; the supplied source does not define a separate detailed ITOM course.

b. Core service workflows

1. **Incident management:** Restore interrupted service and validate the record from creation to closure.
2. **Incident fields:** Check caller, category, impact, urgency, and configured priority calculations.
3. **Assignment:** Verify that the configured rules route work to the correct support group.
4. **Problem management:** Link related incidents and validate investigation of the underlying cause.
5. **Known errors:** Check the configured handling of a documented cause and workaround.
6. **Change types:** Distinguish standard, normal, and emergency changes.
7. **Change control:** Validate required approvals before the configured implementation stage.
8. **Change plans:** Check scheduling, implementation steps, rollback instructions, and closure evidence.

c. Website exercise and evidence

1. **Website example:** Report a failed quiz submission as an incident in the training environment.
2. **Investigation:** Link recurring submission failures to a problem record.
3. **Release example:** Prepare a change record for the website correction.
4. **Evidence:** Submit workflow cases, expected results, record links or screenshots, and a defect report.
5. **Lesson source:** ServiceNow All in One.docx; ITSM, Incident Management, Problem Management, and Change Management sections.

32. ServiceNow catalog requests and record producers

a. Concepts and validation

1. **Service Catalog:** Explain the self-service entry point for requesting available services.
2. **Catalog items:** Check availability and eligibility for the intended users.
3. **Variables:** Validate required inputs, defaults, reference choices, and conditional fields.
4. **REQ:** Identify the overall request record.
5. **RITM:** Identify the requested item and verify its submitted values.
6. **SCTASK:** Verify the fulfillment task and its relationship to the requested item.
7. **Approvals:** Test approval and rejection paths against the configured requirements.
8. **Fulfillment:** Check task assignment and completion before request closure.
9. **Alternative paths:** Test cancellation, missing information, and fulfillment failure.
10. **Record Producer:** Explain a catalog form that creates a record in a target table.
11. **Field mapping:** Check that producer inputs populate the intended target record.
12. **Order guides:** Recognize their place among the catalog entry types listed in the source.

b. Website exercise and evidence

1. **Catalog example:** Request access to the website training environment.
2. **Record Producer example:** Submit a website issue and verify the incident that is created.
3. **Negative test:** Attempt submission without a required value.
4. **Evidence:** Submit a request-to-task trace and a separate producer-to-record mapping check.
5. **Lesson source:** ServiceNow All in One.docx; Service Catalog, Request Management, and interview question 28 on Record Producers.

33. ServiceNow SLAs rules and access

a. Time and workflow behavior

1. **SLA purpose:** Explain the agreed target for response or resolution.
2. **Attachment conditions:** Verify that the correct SLA attaches to the relevant record.
3. **Timer lifecycle:** Test start, pause, resume, and stop conditions individually.
4. **Business schedules:** Verify elapsed time against working hours, holidays, and time zones.
5. **Breach behavior:** Check escalation and notification when the configured limit is exceeded.
6. **Record changes:** Retest timer behavior after priority or state changes.
7. **Example targets:** Treat the handout SLA durations as exercise data, not universal ServiceNow defaults.

b. Rules and protection

1. **Business Rules:** Explain server-side processing when configured record conditions are met.
2. **Client-side behavior:** Explain browser form behavior and distinguish it from server-side validation.
3. **Rule triggers:** Test applicable creation, update, and deletion conditions.
4. **Data integrity:** Verify that rules update the intended values without changing unrelated records.
5. **Notifications:** Verify trigger, recipient, subject, content, and timing.
6. **Role access:** Check allowed and denied operations using representative user roles.
7. **Knowledge access:** Check search results and article visibility for the intended audience.

c. Website exercise and evidence

1. **Website example:** Test assignment and SLA behavior for a website incident using a supplied priority matrix.
2. **Evidence:** Submit a timer worksheet, rule decision cases, and an allowed-versus-denied access matrix.
3. **Lesson source:** ServiceNow All in One.docx; SLA validation, roles and security, notifications, and interview questions 25 to 27.

34. ServiceNow ITAM and CMDB

a. Asset management

1. **ITAM purpose:** Track assets through acquisition, use, transfer, and retirement.
2. **Asset records:** Validate identifiers, serial numbers, ownership, location, and required fields.
3. **Procurement and receiving:** Check that received quantities create the expected inventory records.
4. **Assignment and transfer:** Verify the current user and retained assignment history.
5. **Lifecycle restrictions:** Test permitted transitions and reject invalid use of retired or disposed assets.
6. **Software assets:** Compare license entitlement with recorded consumption in a supplied scenario.

b. Configuration data

1. **CMDB purpose:** Store configuration items and their relationships.
2. **CI classes:** Check that a configuration item uses the intended class and required attributes.
3. **Relationships:** Validate links between the website application, its server, and its database.
4. **Asset versus CI:** Distinguish asset ownership and lifecycle from configuration and service relationships.
5. **Discovery data:** Check incoming updates against expected records and duplicate-handling rules.
6. **Data quality:** Inspect missing values, stale values, duplicate records, and configured audit history.

c. Website exercise and evidence

1. **Asset example:** Follow a synthetic learner laptop from receipt through assignment and retirement.
2. **Website example:** Map the website application to sample infrastructure CIs.
3. **Evidence:** Submit lifecycle test cases, an asset-to-CI mapping, and a relationship diagram.
4. **Lesson source:** ServiceNow All in One.docx; ITAM and CMDB sections, including the CI validation table.

35. ServiceNow HRSD and customer service

a. Employee services

1. **HRSM and HRSD:** Relate the handout HR service-management terminology to HR Service Delivery.
2. **HR services:** Explain onboarding, offboarding, benefits, and employee information requests.
3. **HR cases:** Validate subject person, requested-for information, service, assignment, and state.
4. **HR tasks:** Check generated work and dependencies across HR, IT, and other fulfillment groups.
5. **Confidentiality:** Test permitted case and field access with synthetic employee data.
6. **HR knowledge:** Verify that employees receive appropriate articles without restricted information.

b. Customer services

1. **CSM purpose:** Manage customer support cases from submission through closure.
2. **Customer identity:** Check account and contact associations on each case.
3. **Customer portal:** Validate submission, updates, attachments, and visible status.
4. **Case processing:** Test assignment and permitted state transitions.
5. **Customer isolation:** Verify that one customer cannot access another customer's restricted case.
6. **Communication:** Check applicable SLAs, notifications, and knowledge access.

c. Website exercise and evidence

1. **Employee example:** Trace onboarding tasks that request website training access.
2. **Customer example:** Trace a learner support case about a quiz result.
3. **Evidence:** Submit one HR case flow, one customer case flow, and negative access tests.
4. **Lesson source:** ServiceNow All in One.docx; HRSM, HRSD, and CSM sections, including the customer case validation table.

36. ServiceNow integration testing and assessment

a. Testing approach

1. **Requirements review:** Resolve ambiguous business rules before setting expected results.
2. **Functional tests:** Check each requirement with observable outcomes.
3. **Negative tests:** Challenge missing inputs and invalid workflow transitions.
4. **Integration mapping:** Verify source values, transformations, and target records.
5. **Synchronization:** Check creation and updates across the participating systems.
6. **Failure handling:** Test duplicate submissions and integration errors.
7. **Upgrade testing:** Run smoke checks followed by risk-based regression of affected processes.
8. **End-to-end tests:** Trace the business request through all required stages to closure.

b. Practical assessment

1. **Website scenario:** Trace a website issue through incident, investigation, controlled change, and retest.
2. **Test package:** Submit cases with data, preconditions, steps, and expected results.
3. **Defect evidence:** Record actual behavior, severity, priority, reproduction steps, and supporting evidence.
4. **Interview practice:** Explain each module and a test scenario using the learner's own lab experience.
5. **Training environment:** Use configured training modules or clearly labeled simulations when access is unavailable.
6. **Automation link:** Apply course browser and Karate skills to authorized, available training interfaces.
7. **Delivery scope:** Use these lessons within the suggested course plan; adjust instructor pacing to module access and learner progress.
8. **Lesson source:** ServiceNow All in One.docx; tester interview questions 13 to 24 and module integration-validation sections.

37. Large language models and workplace Copilot

a. What an LLM is and how it works

1. **LLM:** a model that learns language patterns from training data
2. **Tokens:** units used to represent text
3. **Inference:** generating tokens step by step from context
4. **Training:** changing learned model parameters
5. A prompt supplies context; it does not normally retrain the model
6. Context limits and variable outputs
7. Plausible language can still be incorrect
8. Grounding supplies relevant material
9. Retrieval adds selected source content to context
10. Grounding and retrieval do not guarantee accuracy
11. Verify claims, expected results, and code against trusted evidence

b. Copilot in organizations

1. **Microsoft 365 Copilot:** organizational documents and productivity applications
2. **GitHub Copilot:** software development and code assistance
3. Copilot products use models; Copilot is not one universal LLM
4. Features and model access depend on product, licensing, permissions, and organization settings
5. Use employer-approved accounts and tools
6. Check the context being shared
7. Follow organizational data rules
8. Do not assume one license provides every model or all company data
9. Record the product and visible model; mark hidden model details as unknown

38. Copilot practice and verification

a. A simple response flow

1. User task and permitted context
2. product prepares the request
3. model generates a response
4. product presents the result
5. tester verifies the result against requirements and evidence.

b. Hands on workplace exercise

1. Use Microsoft 365 Copilot to summarize a synthetic requirement or prepare a draft test outline.
2. Use GitHub Copilot to draft a small browser test or explain a failure.
3. Compare the result with manually written acceptance criteria, execute generated code, and correct unsupported assertions.
4. Use recorded instructor examples if a license is unavailable.

c. Learning evidence

1. Explain tokens, training, inference, context, grounding, and hallucinations in plain language.
2. Distinguish the Copilot product from the model it uses and describe why available models may vary.
3. **Sanitized prompt:** Record the instruction without sensitive information
4. **Original output:** Preserve the initial AI response
5. **Verification:** Explain how the result was checked
6. **Corrected result:** Present the supported revision
7. Explain one accepted and one rejected suggestion using independent evidence.
8. **Terminology:** LLM means large language model.
9. LMS means learning management system, a different concept used to administer courses and learning records.
10. The AI foundations topic here is LLMs.
11. **Official references:** Microsoft Copilot overview https://learn.microsoft.com/en-us/microsoft-365/copilot/microsoft-365-copilot-overview
12. **GitHub Copilot overview https:**//docs.github.com/en/copilot/get-started/about-github-copilot

39. Improving the existing website for testing

1. The existing website is the shared project.
2. We add or refine test components in small increments as the course advances.
3. The instructor confirms which site and test environment are used for a cohort; the curriculum does not depend on creating a replacement application.

a. Forms

What students add or refine

1. **Required fields:** Reject a submission when mandatory input is missing
2. **Format rules:** Check that input follows the expected pattern
3. **Clear error and success messages:** Explain the result of an attempted action

What students test

1. **Valid and invalid inputs:** Check accepted values and rejected values
2. **Boundaries:** Check values at the edges of a permitted range
3. **Keyboard use:** Complete the interaction without a mouse
4. **Recovery:** Continue successfully after an error

b. Choices and dates

What students add or refine

1. **Dropdowns:** Select one of the available options
2. **Radio buttons:** Choose one option from a group
3. **Checkboxes:** Select or clear independent options
4. **Date constraints:** Enforce the allowed date range

What students test

1. **State changes:** Check behavior after a selection changes
2. **Combinations:** Test interactions between selected options
3. **Default values:** Verify the initial selections
4. **Invalid dates:** Reject dates outside the allowed rules

40. Component details continued

a. Tables

What students add or refine

1. **Search:** Find records matching a query
2. **Sorting:** Arrange records in the requested order
3. **Pagination:** Navigate between groups of records
4. **Filters:** Limit results to matching records
5. **Empty states:** Explain when no records are available

What students test

1. **Expected records:** Verify that the correct items are returned
2. **Ordering:** Check the sequence of displayed records
3. **Counts:** Check reported totals against the data
4. **Combinations:** Test interactions between selected options
5. **No results:** Verify the response to an unmatched query

b. Dynamic feedback

What students add or refine

1. **Loading indicators:** Show that an operation is in progress
2. **Delayed results:** Handle responses that arrive later
3. **Disabled actions:** Prevent actions while their prerequisites are unmet

What students test

1. **Wait conditions:** Continue when the required state is reached
2. **Repeated clicks:** Check behavior when an action is requested again
3. **Timeouts:** Handle an operation that takes too long
4. **Retry behavior:** Check what happens when an operation is repeated

41. Component details continued

a. Browser interactions

What students add or refine

1. **Dialogs:** Handle an interactive prompt
2. **Additional windows:** Switch to the intended browser window
3. **Safe upload examples:** Practice with permitted synthetic files

What students test

1. **Correct context:** Act on the intended frame or window
2. **Cancel paths:** Check the result of abandoning an action
3. **Allowed file rules:** Validate permitted upload types and sizes
4. **Cleanup:** Restore reusable test state

b. Scores or submissions

What students add or refine

1. A synthetic submission flow with visible confirmation

What students test

1. **Data consistency:** Compare values across the page and service
2. **Duplicate submission:** Check that repeated submissions are handled correctly
3. **API responses:** Verify the service response to an action
4. **Error paths:** Check the behavior after a failed request

42. Component details continued

a. Accessibility

What students add or refine

1. **Labels:** Identify the purpose of each user interface control
2. **Visible focus:** Make the active control easy to identify
3. **Readable errors:** Explain validation problems clearly
4. **Responsive layout:** Adapt the interface to different screen sizes

What students test

1. **Keyboard flow:** Follow a usable sequence without a mouse
2. **Focus order:** Move between controls in a logical order
3. **Contrast review:** Check that content can be distinguished visually
4. **Screen sizes:** Review the interface at the selected viewport sizes

b. Optional AI hint

What students add or refine

1. A bounded hint or explanation with recorded or live responses

What students test

1. **Grounding:** Check the use of supplied sources
2. **Usefulness:** Assess whether the response helps complete the task
3. **Unavailable response:** Check behavior when the AI service cannot answer
4. **Safe fallback:** Provide an appropriate alternative after failure

43. Project scope and milestones

a. Suggested minimum project scope

1. Choose one form or input improvement
2. Choose one data or dynamic component improvement
3. Choose one quality or error-handling improvement
4. Use instructor scaffolding to keep the project focused on testing
5. An AI-backed component is optional
6. A reviewed AI-assisted testing exercise is required

b. Project milestones

c. Week 25

1. Baseline existing behavior
2. Agree on component changes
3. Write acceptance criteria and risks
4. Prepare test data and cases

d. Weeks 26 and 27

1. Implement the agreed improvements
2. Execute manual tests
3. Add focused automation
4. Investigate defects
5. Connect the suite to the course pipeline

e. Week 28

1. Retest fixes
2. Run regression
3. Present test evidence
4. Demonstrate the website
5. Hand over the test package and remaining findings
6. Use resettable synthetic data and an approved test environment.
7. Keep deliberately defective examples clearly identified in the lab environment.
8. Shared-site deployment follows the instructor’s existing release process.

44. Assessments and completion requirements

1. Assessment rewards demonstrated testing judgment and reproducible work.
2. The proposed weighting below can be adjusted before a cohort begins.
3. Publish the final rubric and allow learners to correct work after feedback.

a. Topic quizzes

1. **Weight:** 15%

Evidence

1. 25-question reviews with explanations and follow-up on weak topics

b. Practical labs

1. **Weight:** 30%

Evidence

1. Manual QA
2. Java and JS or TS
3. Three UI tools
4. Karate
5. DevOps and ServiceNow labs

c. Reviews and communication

1. **Weight:** 10%

Evidence

1. **Peer review:** Evaluate another person's change
2. **Defect triage:** Decide how reported issues will be handled
3. **Concise summaries:** Communicate the essential testing findings
4. **Clear technical explanations:** Explain decisions using understandable evidence

45. Assessment details continued

a. AI evaluation portfolio

1. **Weight:** 15%

Evidence

1. **Prompt/output review:** Check the AI response against its task and sources
2. **Rejected suggestions:** Explain why an AI suggestion was not accepted
3. **Evaluation cases:** Provide inputs used to assess AI behavior
4. **Error analysis:** Explain why observed outcomes were incorrect

b. Existing website project

1. **Weight:** 30%

Evidence

1. **Component improvements:** Demonstrate the changes made to the existing website
2. **Traceable testing:** Connect requirements to tests and results
3. **Automation:** Show repeatable checks executed by the test tools
4. **Results:** Record observed outcomes and their supporting evidence
5. **Demonstration:** Show the completed work in operation

c. Proposed completion standard

1. Achieve at least 70% overall and complete every required practical checkpoint.
2. Achieve at least 70% on the final project rubric and demonstrate the submitted work independently.
3. Provide reproducible evidence and disclose AI assistance.
4. Unsupported results or copied work without understanding do not satisfy a practical requirement.
5. Record unresolved defects and limitations honestly.
6. Course completion does not require pretending that an application has no defects.

46. Final project rubric and portfolio

a. Final project rubric

b. Scope and traceability

1. **Points:** 15

What a successful submission shows

1. **Clear criteria:** State measurable expected outcomes
2. **Prioritized risks:** Order testing by the significance of failure
3. **Links from requirements to cases:** Show which test covers each requirement

c. Test design and manual execution

1. **Points:** 20

What a successful submission shows

1. **Positive test coverage:** Demonstrate valid user behavior
2. **Negative test coverage:** Demonstrate invalid user behavior
3. **Boundary test coverage:** Demonstrate checks at input limits
4. **Exploratory test coverage:** Document focused investigative testing

d. Automation and maintainability

1. **Points:** 25

What a successful submission shows

1. **Reliable assertions:** Check outcomes that establish the intended behavior
2. **Reusable structure:** Share interaction code without obscuring test intent
3. **Controlled data:** Use known inputs and reset state between runs
4. **Readable failures:** Provide enough context to diagnose a failed check

47. Criterion details continued

a. API and pipeline evidence

1. **Points:** 15

What a successful submission shows

1. **Repeatable commands:** Document how another person can rerun the suite
2. **Integration checks:** Verify interactions between application components
3. **Reports:** Summarize the execution results
4. **Demonstrated failure handling:** Show how the workflow responds to a failed check

b. Quality and AI judgment

1. **Points:** 15

What a successful submission shows

1. Relevant nonfunctional checks and a verified AI-assisted artifact or AI evaluation

c. Demonstration and handover

1. **Points:** 10

What a successful submission shows

1. **Clear walkthrough:** Explain the project and its evidence step by step
2. **Reproducible defects:** Provide enough detail to recreate each failure
3. **Setup instructions:** Explain how to prepare the test environment
4. **Limitations:** State unresolved issues and coverage gaps

48. Required portfolio

1. Feature map and acceptance criteria
2. Test plan and traceability matrix
3. Test cases, execution results, and defect reports
4. Java and JavaScript or TypeScript exercises
5. Selenium, Playwright, and Cypress examples
6. Karate API suite and Jenkins evidence
7. Docker and Kubernetes lab evidence
8. AWS environment map and ServiceNow tester validation package
9. Copilot review
10. Final demonstration and handover notes

49. Teaching resources and curriculum coverage

Curriculum requirement	Where it is taught	Evidence
Early quality assurance	Modules 1 to 3	Reviewed requirements and testable acceptance criteria
Manual testing and QA practice	Modules 3 to 6	Test plan, designed cases, execution and defects
Coding and test automation	Modules 7 to 11	Java and JS or TS, three browser tools, Karate suite
Delivery and quality reporting	Modules 12 and 13	Jenkins, container, cloud and ServiceNow evidence
AI literacy and responsible use	Modules 14 and 15	LLM explanation and reviewed Copilot artifacts
ML concepts and AI evaluation	Modules 16 and 17	Metrics, evaluation dataset and error analysis
End to end practical application	Module 18	Improvements and test evidence for the existing site

50. Teaching tools and resources

a. Tools and resources in the Academy

1. **Learning materials:** notes, mini quizzes, flashcards, carousels, and labs
2. **Languages:** Java and JavaScript or TypeScript
3. **Browser automation:** Selenium, Playwright, and Cypress
4. **API automation:** Karate
5. **Data investigation:** SQL and browser developer tools
6. **Collaboration:** Git and GitHub
7. **Execution:** Java test runner, build tool, Node.js, and npm
8. **Continuous integration:** Jenkins
9. **Machine learning:** guided notebook or prepared worksheet
10. Google AI Studio and Gemini can support prompt and evaluation lessons where instructor access is available.
11. Existing Google Docs or Sheets materials support lesson preparation.
12. Firebase can provide a teaching example of score storage and data validation.
13. These are integration options from the Academy scope; this curriculum does not assert that each integration is already active.

b. Academy portal

1. https://jzscodegame.github.io/SDLC---STLC/

c. Existing automation playground

1. https://atom-php.onrender.com/index.php
2. The saved integration report identifies the Render site as an automation playground.
3. Use the existing site selected by the instructor for the component-improvement project.
4. Check availability and lab access before class; a local copy, fixture, or mock can support the same objective when needed.
5. Private administration consoles are not student links.

51. Instructor preparation and upkeep

1. **Component baseline:** Prepare the initial website behavior
2. **Expected outcomes:** Define what the lesson should demonstrate
3. **Synthetic data:** Supply resettable practice inputs
4. **Starter materials:** Prepare the lab resources
5. **Answer explanations:** Verify the reasoning behind quiz answers
6. **Assessment rubric:** Publish the evaluation criteria
7. Review AI-generated content against its lesson sources.
8. Date and version materials so results can be interpreted against what students actually saw.
9. Use quiz and lab feedback to select review topics.
10. Refreshing a dated quiz does not guarantee a different set of questions when the bank is small.
11. Preserve the previous usable lesson package and test environment when an update fails.
12. Confirm student-visible behavior after an approved release.